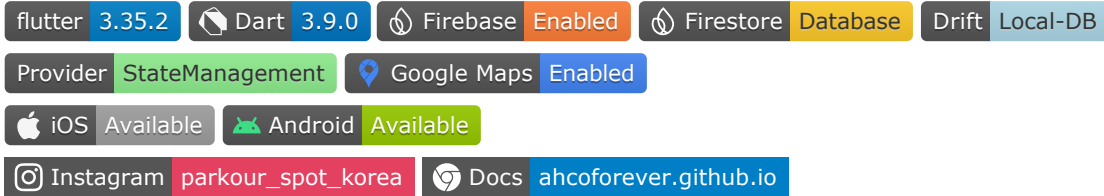


PARKOUR SPOT



한국의 파쿠르 스팟을 찾고 공유하는 Flutter 앱

목차

1. 프로젝트 개요
2. 문제 정의 & 해결 방안
3. 핵심 기능
4. 기술 스택 & 아키텍처
5. 핵심 구현 - 지도 시스템
6. 핵심 구현 - 실시간 위치 추적
7. 데이터 아키텍처
8. MVVM 패턴 구현
9. 성능 최적화 & 사용자 경험
10. 프로젝트 성과 & 향후 계획
11. 시작하기

프로젝트 개요

Parkour Spot Korea

파쿠르라는 스포츠는 도시를 새로운 시각으로 바라보며 창의적인 움직임을 스스로 발견하고 만들어가는 운동입니다. 마치 도시가 거대한 놀이터가 되는 것처럼, 평범한 계단과 벽, 난간이 창의적인 움직임의 캔버스가 되는 순간을 경험할 수 있습니다. 하지만 한국에서 파쿠르를 즐기려는 사람들이 직면하는 가장 큰 어려움은 바로 '어디서 연습해야 하는가?'라는 것입니다. 이 프로젝트는 바로 그 질문으로부터 시작되었습니다.

파쿠르 스팟(parkour spot)은 한국 전역의 파쿠르 연습 장소를 발견하고, 기록할 수 있는 지도 기반 모바일 애플리케이션입니다. 더 나아가 사용자가 방문한 지역을 육각형 그리드로 시각화하여, 자신만의 '스크래치 맵'을 완성해나가는 게이미피케이션(Gamification) 요소를 더했습니다.

핵심 가치

- 실용적인 위치 기반 서비스 구현
- 모바일 퍼스트 사용자 경험
- Clean Architecture 기반 확장 가능한 설계

개발 환경: Flutter 3.8.1, Firebase, Google Maps API

플랫폼: iOS, Android

문제 정의 & 해결 방안

해결하고자 한 문제

- 파쿠르 연습 장소 정보의 분산화
- 지역별 스팟 접근성 부족
- 개인 방문 기록 관리의 어려움

해결 방안

- 지도 기반 통합 플랫폼 구축
- 위치 기반 실시간 검색 시스템
- 개인화된 방문 추적 (Scratch Map)
- 소셜 인증 기반 커뮤니티 형성

타겟 사용자

- 파쿠르 초보자 ~ 전문가
- 새로운 연습 장소를 찾는 사용자
- 개인 성취도를 추적하고 싶은 사용자

핵심 기능

주요 기능 4가지

1. 인터랙티브 지도 - 완전한 구현

```
/// 중심 좌표 기준 주변 스팟을 가져와 마커로 표시
Future<void> loadAndShowSpots(LatLng center, {double radiusKm = 5.0})
async {
  if (_isLoadingSpots) return; // 중복 호출 방지
  _isLoadingSpots = true;
  notifyListeners();

  try {
    // Firebase에서 주변 스팟 데이터 fetch
    final spots = await _spotRepo.fetchNearby(
      center: center,
      radiusKm: radiusKm,
    );

    // 스팟 정보 캐시에 저장 (빠른 조회를 위해)
    _loadedSpots.clear();
    for (final spot in spots) {
      _loadedSpots[spot.documentId] = spot;
    }
  }
}
```

```

// 마커 동적 생성 및 교체
_parkourMarkers
  ..clear() // 기존 마커 완전 제거
  ..addAll(
    spots.map((spot) {
      return Marker(
        markerId: MarkerId('spot_${spot.documentId}'),
        position: spot.location,
        infoWindow: InfoWindow(
          title: spot.displayName.isNotEmpty
            ? spot.displayName : spot.name,
          snippet: spot.description,
        ),
        icon: _getMarkerIcon(spot.category), // 카테고리별 색상
        onTap: () {
          print('마커 탭: ${spot.name} (${spot.documentId})');
          _onMarkerTapped(spot);
        },
      );
    }),
  );

print('주변 스팟 ${spots.length}개 로드/표시 완료');
} catch (e) {
  print('loadAndShowSpots 실패: $e');
} finally {
  _isLoadingSpots = false;
  notifyListeners();
}
}

```

2. 지능형 검색 시스템

```

/// 검색/기본 마커 동적 전환
Set<Marker> _getDisplayMarkers(ScratchMapViewModel viewModel) {
  if (_showSearchResults && _searchViewModel.searchResults.isNotEmpty) {
    // 검색 결과 마커 (빨간색으로 강조)
    return _searchViewModel.searchResults.map((spot) {
      return Marker(
        markerId: MarkerId('search_${spot.documentId}'),
        position: spot.location,
        infoWindow: InfoWindow(title: spot.name, snippet: spot.address),
        icon:
          BitmapDescriptor.defaultMarkerWithHue(BitmapDescriptor.hueRed),
        onTap: () => _showSpotBottomSheet(spot),
      );
    }).toSet();
  }
  // 기본 마커 (카테고리별 색상)
  return viewModel.parkourMarkers;
}

```

3. 스크래치 맵 (방문 추적)

- 위치 스트림 기반 실시간 hexagon 생성
- 방문한 지역 시각화 및 성취도 측정
- Firebase 동기화로 기기간 데이터 공유

4. Google OAuth 2.0

- iOS/Android 통합 소셜 로그인
- 개인화된 북마크 및 방문 기록 관리

기술 스택 & 아키텍처

Tech Stack

Frontend

```
Flutter/Dart + Provider + Material Design 3
```

Backend & Database

```

Firebase Ecosystem
├── Authentication (Google OAuth)
├── Firestore (실시간 DB)
├── Storage (이미지)
├── Remote Config

Local Storage
├── SQLite (Drift ORM)
```

Maps & Location

```
Google Maps Flutter + Geolocator + Geocoding
```

Clean Architecture

```

Presentation Layer (UI + ViewModels)
      ↓
Business Logic Layer (Repositories + Services)
      ↓
Data Layer (Firebase + Local Database)
```

프로젝트 구조

```
lib/
├── core/           # 핵심 설정 및 유틸리티
├── database/       # Drift 데이터베이스 스키마
├── interfaces/     # Repository 인터페이스
├── repositories/   # 데이터 액세스 레이어
├── routes/         # 앱 라우팅 설정
├── screens/        # UI 화면들
│   ├── spot/      # 스팟 관련 화면
│   └── ...
├── services/       # 비즈니스 로직 서비스
├── theme/          # 앱 테마 설정
├── viewmodel/      # MVVM 패턴의 뷰모델
└── main.dart       # 앱 진입점
```

핵심 구현 - 지도 시스템

Google Maps API 고급 활용

카메라 이동 기반 동적 로딩

```
onCameraIdle: () async {
  if (mapController == null) return;
  final size = MediaQuery.of(context).size;

  // 화면 중심점 계산
  final center = await mapController!.getLatLng(
    ScreenCoordinate(
      x: (size.width ~/ 2),
      y: (size.height ~/ 2),
    ),
  );

  // 새로운 중심점 기준 스팟 재로드 (5km 반경)
  viewModel.loadAndShowSpots(center, radiusKm: 5);
}
```

다크 테마 맵 스타일 적용

```
// 앱 시작 시 커스텀 맵 스타일 로드
rootBundle.loadString('assets/map_style/map_style.json')
  .then((style) {
    setState(() {
      _mapStyle = style; // GoogleMap의 style 속성에 적용
    });
  }).catchError((e) {
```

```
debugPrint('Map style load failed: $e');
});
```

카테고리별 커스텀 마커 아이콘

```
BitmapDescriptor _getMarkerIcon(String category) {
  switch (category.toLowerCase()) {
    case 'park':
      return BitmapDescriptor.defaultMarkerWithHue(
        BitmapDescriptor.hueGreen);
    case 'school':
      return BitmapDescriptor.defaultMarkerWithHue(
        BitmapDescriptor.hueBlue);
    case 'parkour_gym':
      return BitmapDescriptor.defaultMarkerWithHue(
        BitmapDescriptor.hueOrange);
    default:
      return BitmapDescriptor.defaultMarkerWithHue(180.0);
  }
}
```

핵심 구현 - 실시간 위치 추적

위치 기반 서비스

실시간 위치 스트림

```
void _startLocationTracking() {
  _positionStreamSubscription = _locationRepository
    .getPositionStream()
    .listen((Position position) {
      _handleLocationUpdate(position);
    });
}

Future<void> _handleLocationUpdate(Position position) async {
  final current = LatLng(position.latitude, position.longitude);
  final currentHexId = await _locationRepository.generateHexId(current);

  // 새로운 영역 진입 시 헥사곤 자동 생성
  if (_lastHexagonId != currentHexId) {
    await _createHexagonAtPosition(current);
    _lastHexagonId = currentHexId;
  }
}
```

위치 권한 관리

```
Future<Position?> getCurrentPosition() async {
  try {
    // 위치 권한 확인
    LocationPermission permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied) {
        throw Exception('위치 권한이 거부되었습니다');
      }
    }
  }

  // 고정밀 위치 가져오기
  return await Geolocator.getCurrentPosition(
    locationSettings: LocationSettings(
      accuracy: LocationAccuracy.high,
      timeLimit: Duration(seconds: 5),
    ),
  );
} catch (e) {
  print('위치 가져오기 실패: $e');
  return null;
}
```

데이터 아키텍처

하이브리드 저장 전략

Firebase + Local DB 동기화

```
class ParkourSpotRepositoryWrapper implements IParkourSpotRepository {
  final FirestoreParkourSpotRepository _firebaseRepo;
  final LocalParkourSpotRepository _localRepo;

  @override
  Future<List<ParkourSpot>> fetchNearby({
    required LatLng center,
    required double radiusKm,
  }) async {
    try {
      // 1. 로컬 캐시 먼저 확인
      final localSpots = await _localRepo.fetchNearby(
        center: center, radiusKm: radiusKm);

      // 2. Firebase에서 최신 데이터 가져와 캐시 업데이트
      final remoteSpots = await _firebaseRepo.fetchNearby(
        center: center, radiusKm: radiusKm);
      await _localRepo.cacheSpots(remoteSpots);
    }
  }
}
```

```

        return remoteSpots.isEmpty ? localSpots : remoteSpots;
    } catch (e) {
        return await _localRepo.fetchNearby(center: center, radiusKm:
radiusKm);
    }
}
}
}

```

데이터 흐름

```

UI → ViewModel → Repository → Service → Database
    ^           ^           ^           ^
    Provider State Interface

```

MVVM 패턴 구현

상태 관리 & 비즈니스 로직

ViewModel 설계

```

class ScratchMapViewModel extends ChangeNotifier {
    final IScratchMapRepository _scratchMapRepository;
    final IParkourSpotRepository _spotRepo;

    // 상태 관리
    ScratchMapState _state = ScratchMapState.initial();
    final Set<Marker> _parkourMarkers = {};

    // 비즈니스 로직
    Future<void> loadAndShowSpots(LatLng center, {double radiusKm = 5.0})
    async {
        _isLoadingSpots = true;
        notifyListeners();

        try {
            final spots = await _spotRepo.fetchNearby(center: center, radiusKm:
radiusKm);
            _updateMarkersFromSpots(spots);
        } finally {
            _isLoadingSpots = false;
            notifyListeners();
        }
    }

    // UI 콜백 처리
    Function(ParkourSpot)? onSpotMarkerTapped;
}

```


반응형 UI

- Provider 기반 상태 변경 감지
- 선언적 UI 업데이트
- 에러 상태 처리 및 복구

성능 최적화 & 사용자 경험

성능 최적화 전략

1. 네트워크 최적화

- Repository 패턴으로 캐싱 레이어 구현
- 오프라인 우선 데이터 로딩
- 시간 초과(timeout) 처리로 사용자 경험 개선

2. 메모리 관리

```
@override
void dispose() {
  _stopLocationTracking(); // 위치 스트림 정리
  _searchViewModel.dispose(); // ViewModel 리소스 해제
  mapController?.dispose(); // 지도 컨트롤러 정리
  super.dispose();
}
```

3. UX 개선

- 로딩 상태별 인디케이터 표시
- 검색/기본 모드 부드러운 전환
- 에러 상황 사용자 친화적 처리

코드 품질 지표

정적 분석 지표

- Dart Analyzer 100% 통과
- flutter_lints 규칙 완전 준수
- 코드 복잡도 최적화

구조적 품질

- Clean Architecture 완전 구현
- 40+ Repository/Service 클래스 모듈화
- Interface 기반 의존성 주입

성능 지표

- 중복 로딩 방지: `_isLoadingSpots` 플래그

- 스팟 캐싱: `_loadedSpots` 맵으로 빠른 조회
 - 배경 기반 로딩: 필요한 데이터만 효율적으로 fetch
 - 상태 관리: Provider 기반 반응형 UI 업데이트
-

프로젝트 성과 & 향후 계획

개발 성과

기술적 성과

- **Clean Architecture** 완전 구현
- **13개 주요 화면** 및 **40+ 클래스** 체계적 설계
- **Firebase** 실시간 동기화 안정적 구현
- **Google Maps API** 고급 기능 활용

사용자 가치

- 직관적인 지도 기반 인터페이스
- 실시간 위치 추적 및 스팟 발견
- 개인화된 탐험 기록 시각화

향후 개발 계획

기능 확장

- 사용자 리뷰 & 평점 시스템
- 실시간 채팅 및 모임 기능
- 스팟 이미지/영상 업로드
- 파쿠르 커뮤니티

기술적 개선

- 성능 모니터링 대시보드
- CI/CD 파이프라인 구축
- 단위 테스트 커버리지 확대

개발자로서의 학습

- 대규모 Flutter 앱 아키텍처 설계 경험
 - Firebase 백엔드 서비스 전체 스택 활용
 - 사용자 중심 UX/UI 설계 역량 개발
-

시작하기

필수 조건

- Flutter SDK (^3.8.1)
- Dart SDK
- Android Studio / VS Code

- iOS 개발을 위한 Xcode (macOS)

설치

1. 저장소 클론

```
git clone https://github.com/yourusername/parkourspotkorea.git
cd parkourspotkorea
```

2. 의존성 설치

```
flutter pub get
```

3. Firebase 설정

- Firebase 프로젝트 생성
- `google-services.json` (Android) 및 `GoogleService-Info.plist` (iOS) 파일 추가
- Firebase Authentication, Firestore, Storage 서비스 활성화

4. Google Maps API 설정

- Google Cloud Console에서 Maps SDK 활성화
- API 키를 Android/iOS 설정에 추가

5. 앱 실행

```
flutter run
```

기여하기

1. 이 저장소를 Fork 합니다
2. feature 브랜치를 생성합니다 (`git checkout -b feature/AmazingFeature`)
3. 변경사항을 커밋합니다 (`git commit -m 'Add some AmazingFeature'`)
4. 브랜치에 Push 합니다 (`git push origin feature/AmazingFeature`)
5. Pull Request를 생성합니다

문의

프로젝트에 대한 문의사항이나 제안이 있으시면 이슈를 생성해 주세요.
